

```

# =====
# PRESENTED BY DHANUSH P
# ADVANCED MATHEMATICS COURSE PROJECT
# TOPIC: Advanced SPAM SMS AND MAIL DETECTION
# 2026 BRANCH (1ST YEAR MTECH)

# Mathematical Concepts:
# • Bayes' Theorem
# • Prior Probability
# • Posterior Probability
# • TF-IDF Vectorization
# • Laplace Smoothing
# • Log-Likelihood
# • Information Entropy
# • Explainable AI (Word Contributions)
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import re

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB

# -----
# Load Dataset
# -----

url = "https://raw.githubusercontent.com/justmarkham/pycon-2016-tutorial/master/data/sms.tsv"

df = pd.read_table(url, header=None, names=["label", "message"])

# -----
# Text Preprocessing
# -----

def clean_text(text):
    text=text.lower()
    text=re.sub(r"^[a-zA-Z\s]", "", text)
    text=re.sub(r"\s+", " ", text)
    return text.strip()

df["clean_message"]=df["message"].apply(clean_text)
df["label_num"]=df["label"].map({"ham":0, "spam":1})

# -----
# TF-IDF
#
#  $TF(t, d) = f(t, d) / \sum f$ 
#  $IDF = \log(N / (df + 1))$ 
#  $TF-IDF = TF \times IDF$ 
# -----

vectorizer=TfidfVectorizer(
    stop_words="english",

```

```

        max_features=5000
    )

X=vectorizer.fit_transform(df["clean_message"])
y=df["label_num"]

# -----
# Train-Test Split
# -----

X_train,X_test,y_train,y_test=train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# -----
# Train Multinomial Naive Bayes
#
#  $P(C|X)=P(X|C)P(C)/P(X)$ 
# -----

model=MultinomialNB(alpha=1.0)
model.fit(X_train,y_train)

# -----
# Priors
# -----

ham_prior=(df["label"]=="ham").mean()
spam_prior=(df["label"]=="spam").mean()

feature_names=vectorizer.get_feature_names_out()

# -----
# Prediction Function
# -----

def predict_message(message):

    cleaned=clean_text(message)
    vector=vectorizer.transform([cleaned])

    prediction=model.predict(vector)[0]
    probability=model.predict_proba(vector)[0]
    log_prob=model.predict_log_proba(vector)[0]

    ham=probability[0]*100
    spam=probability[1]*100
    confidence=max(ham,spam)

    entropy=-np.sum(probability*np.log2(np.clip(probability,1e-10,1)))

    print("="*70)
    print("          ADVANCED SPAM SMS DETECTION USING BAYES THEOREM")
    print("="*70)

```

```

print("\nMessage:")
print(message)

print("\nPrediction:")

if prediction==1:
    print("🔥 SPAM MESSAGE")
else:
    print("✅ HAM (NOT SPAM)")

print("\nPosterior Probabilities")
print(f"P(Ham|X) : {ham:.2f}%")
print(f"P(Spam|X) : {spam:.2f}%")

print("\nBayesian Statistics")
print(f"P(Ham) = {ham_prior:.4f}")
print(f"P(Spam) = {spam_prior:.4f}")
print(f"Entropy = {entropy:.4f} bits")
print(f"Confidence = {confidence:.2f}%")
print(f"log P(Ham|X) = {log_prob[0]:.4f}")
print(f"log P(Spam|X) = {log_prob[1]:.4f}")

# =====
# CHART 1
# Posterior Probability Bar Chart
# =====

plt.figure(figsize=(6,4))

bars=plt.bar(
    ["Ham", "Spam"],
    [ham, spam],
    color=["green", "red"]
)

plt.ylim(0,100)
plt.ylabel("Posterior Probability (%)")
plt.title("Bayesian Posterior Probability")

for b in bars:
    plt.text(
        b.get_x()+b.get_width()/2,
        b.get_height()+2,
        f"{b.get_height():.1f}%",
        ha="center",
        weight="bold"
    )

plt.show()

# =====
# CHART 2
# Bayesian Confidence Gauge
# =====

plt.figure(figsize=(8,1.8))

```

```

gradient=np.linspace(0,100,400).reshape(1,-1)

plt.imshow(
    gradient,
    cmap="RdYlGn_r",
    aspect="auto",
    extent=[0,100,0,1]
)

plt.axvline(spam,color="black",linewidth=3)

plt.yticks([])
plt.xticks(range(0,101,10))

plt.xlabel("Spam Probability (%)")
plt.title("Bayesian Decision Confidence Gauge")

plt.show()

# =====
# CHART 3
# Explainable AI
# Top Influential Words
# =====

words=cleaned.split()

contributions=[]

for word in words:

    if word in vectorizer.vocabulary_:

        idx=vectorizer.vocabulary_[word]

        score=model.feature_log_prob_[1][idx]-
model.feature_log_prob_[0][idx]

        contributions.append((word,score))

if contributions:

    contributions=sorted(contributions,key=lambda
x:abs(x[1]),reverse=True)

    top=contributions[:10]

    labels=[x[0] for x in top]
    values=[x[1] for x in top]

    colors=["red" if v>0 else "green" for v in values]

    plt.figure(figsize=(8,5))

    plt.barh(labels,values,color=colors)

    plt.axvline(0,color="black")

```

```

plt.xlabel("Log Probability Contribution")

plt.title("Top Influential Words in Prediction")

plt.gca().invert_yaxis()

plt.show()

# =====
# CHART 4
# Decision Metrics Radar Chart
# =====

categories=[
    "Confidence",
    "Ham Prior",
    "Spam Prior",
    "Posterior",
    "Certainty"
]

certainty=(1-entropy)*100
certainty=max(0,min(certainty,100))

values=[
    confidence,
    ham_prior*100,
    spam_prior*100,
    max(ham,spam),
    certainty
]

values+=values[:1]

angles=np.linspace(0,2*np.pi,len(categories),endpoint=False).tolist()
angles+=angles[:1]

fig=plt.figure(figsize=(6,6))
ax=plt.subplot(111,polar=True)

ax.plot(angles,values,linewidth=2)
ax.fill(angles,values,alpha=0.25)

ax.set_xticks(angles[:-1])
ax.set_xticklabels(categories)

ax.set_ylim(0,100)

plt.title("Bayesian Decision Metrics")

plt.show()

# -----
# User Input
# -----

print("="*70)
print("          ADVANCED SPAM SMS DETECTION USING BAYES THEOREM")

```

```
print("="*70)
msg=input("\nEnter your SMS message:\n\n")
predict_message(msg)
```